

Vamos montar isso como se fosse um pequeno “sistema operacional do DACM”, não só um script.

Vou dividir em:

1. Visão geral
2. Arquitetura em camadas (harness + agents + storage)
3. Modelo de dados (runs, tasks, checkpoints, logs, artifacts)
4. Ciclo de execução passo a passo
5. Lógica “inteligente/autônoma”
6. Esqueleto de implementação em Python (classes e pseudocódigo)
7. Como plugar isso no seu pipeline DACM/DACM Duplo/DACM-G

1. Visão geral

Objetivo: criar um long-running agent que:

controla todo o pipeline DACM (incluindo DACM Duplo) de ponta a ponta;

persiste estado entre execuções (para não depender de contexto da IA);

usa checkpoints para retomar de onde parou;

mantém logs estruturados de tudo que fez e decidiu;

gera exports automáticos (CSV, XLSX, figuras, relatórios parciais);

toma decisões autônomas sobre:

qual canal/processo executar em seguida;

quando reprocessar com parâmetros alternativos;

quando disparar análises agregadas.

Tudo isso com uma “casca” (harness) que garante:

idempotência (rodar duas vezes não estraga nada);

reprodutibilidade (todos os parâmetros e caminhos ficam registrados);

rastreabilidade (cada resultado está associado a um run_id e task_id).

2. Arquitetura em camadas

2.1. Componentes principais

1. Harness / Orquestrador

Processo principal que:

lê configurações globais (dataset, parâmetros DACM padrão, etc.);

gerencia a fila de tarefas (jobs por canal/segmento);

consulta o estado (checkpoints) e decide o próximo passo;

chama os “agents” específicos (setup, processamento, export).

2. Agents especializados

SetupAgent (Inicializador)

Indexa os dados (ex.: paciente chb01, listar 23 canais, localizar crises, etc.).

Calcula métricas globais iniciais (ruído, entropia, etc.) para ranquear canais.

Cria a estrutura de diretórios e o registro inicial de Run.

ProcessingAgent (Executor DACM)

Executa o pipeline DACM/DACM Duplo para um canal/segmento específico.

Gera artefatos: espectrogramas, entropia, energia, métricas SNR/SCI, etc.

Atualiza checkpoints e logs.

AggregationAgent (Exporter/Relatórios)

Lê todos os resultados já disponíveis.

Gera CSV/XLSX agregados, tabelas de ranking, figuras resumo.

Produz relatórios parciais/finais (ex.: por paciente, por canal, por método).

3. State Store (Persistência de estado)

Pode ser:

diretório com arquivos JSON/YAML, ou

SQLite simples.

Armazena:

Runs, Tasks, Checkpoints, Parâmetros utilizados, Métricas globais.

4. Artifact Store

Diretório organizado em:

/runs/{run_id}/raw/

/runs/{run_id}/processed/

/runs/{run_id}/figures/

/runs/{run_id}/exports/ (CSV, XLSX, etc.)

5. Logger

Log estruturado (JSONL) por run:

/runs/{run_id}/logs.jsonl

Cada linha: timestamp, nível (INFO/WARN/ERROR), run_id, task_id, mensagem, dados extras (params, métricas).

6. Hooks de Notificação (opcional)

Pontos de integração para:

mandar e-mail quando terminar paciente;

sinalizar quando ocorrer erro crítico;

atualizar um dashboard.

3. Modelo de dados

3.1. Run

Representa um experimento/benchmark completo (ex.: "DACM_chb01_full").

Campos (exemplo):

```
{
  "run_id": "dacm_chb01_2025-11-27T12-00-00",
  "dataset": "CHB-MIT",
  "subject": "chb01",
  "methods": ["DACM_Duplo", "STFT", "Wavelet", "HHT"],
  "dacm_params_default": {
    "delta_f": 0.5,
    "f_mod": 1.3,
```

```

    "alpha": 0.1
  },
  "status": "running", // "pending", "running", "completed", "failed"
  "created_at": "...",
  "updated_at": "...",
  "meta": { "notes": "Modelo oficial  $\Delta f=0.5$ ,  $f_{\text{mod}}=1.3$ ,  $\alpha=0.1$ " }
}

```

3.2. Task

Representa uma unidade de trabalho (ex.: “processar canal T3–T5 com DACM Duplo”).

```

{
  "task_id": "task_chb01_T3-T5_DACM_Duplo",
  "run_id": "dacm_chb01_2025-11-27T12-00-00",
  "type": "PROCESS_CHANNEL", // ou "SETUP", "AGGREGATION"
  "channel": "T3-T5",
  "segment": "full", // ou intervalo temporal
  "status": "pending", // "pending", "running", "done", "failed"
  "priority": 0.91, // ex.: baseado no ruído/entropia
  "created_at": "...",
  "updated_at": "...",
  "meta": {
    "noise_score": 0.91,
    "ictal_presence": true
  }
}

```

3.3. Checkpoint

Reflete o progresso dentro de uma task ou do run.

```

{
  "checkpoint_id": "chk_chb01_T3-T5_step3",
  "run_id": "dacm_chb01_2025-11-27T12-00-00",
  "task_id": "task_chb01_T3-T5_DACM_Duplo",
  "stage": "DACM_DUO_FEATURES", // nomes de etapas bem definidos
  "status": "completed", // "in_progress", "completed", "failed"
  "timestamp": "...",
  "artifacts": [
    "processed/chb01_T3-T5_DACM_duplo.npy",
    "figures/chb01_T3-T5_DACM_spectrogram.png"
  ],
  "metrics": {
    "entropy_mean": 1.23,
    "entropy_peak": 2.45,
    "snr": 7.8,
    "sci": 0.67
  }
}

```

```

},
"params": {
  "delta_f": 0.5,
  "f_mod": 1.3,
  "alpha": 0.1
}
}

```

3.4. Log Event

```

{
  "timestamp": "...",
  "level": "INFO",
  "run_id": "dacm_chb01_2025-11-27T12-00-00",
  "task_id": "task_chb01_T3-T5_DACM_Duplo",
  "message": "DACM Duplo concluído para canal T3-T5",
  "data": {
    "duration_sec": 32.4,
    "entropy_peak": 2.45
  }
}

```

4. Ciclo de execução

4.1. Fase 1 – SetupAgent (Inicialização do run)

1. Criar run_id e diretórios base.

2. Ler metadados do dataset (CHB-MIT, etc.).

3. Para cada canal:

Estimar:

ruído (ex.: entropia de alta frequência);

presença de crises/microinstabilidades (usando labels se existirem);

desvios espectrais.

Calcular um score de prioridade (com seus pesos: ruído moderado/alto + microinstabilidades + bordas de crises, etc.).

4. Gerar uma lista de Tasks PROCESS_CHANNEL com prioridade definida.

5. Criar um checkpoint global do tipo SETUP_COMPLETED com:

tabela de canais + scores;

parâmetros DACM padrão.

4.2. Fase 2 – ProcessingAgent (loop principal)

Harness faz:

1. Ler estado: tasks pendentes.

2. Escolher próxima task:

status == pending

maior priority

3. Marcar como running.

4. Chamar o ProcessingAgent:

carregar o arquivo de EEG;

extrair segmento (ictal, interictal, etc., se definido);

rodar pipeline DACM Duplo:

DACM passo 1

DACM passo 2

extrair métricas;

opcional: rodar métodos tradicionais para comparação (STFT, Wavelet, HHT).

5. Salvar artefatos (numpy, figuras, CSV por canal).

6. Criar/atualizar Checkpoint da etapa máxima alcançada.

7. Marcar task done ou failed.

8. Voltar ao passo 1 até não haver tasks pendentes.

4.3. Fase 3 – AggregationAgent (exports e relatórios)

Quando:

não há mais tasks pendentes ou

foi atingido algum critério (ex.: 10 canais concluídos, ou um tempo X de execução),

o Harness dispara o AggregationAgent:

1. Ler todos checkpoints status=completed.

2. Montar tabelas agregadas:

por canal:

entropia média/pico DACM vs métodos;

SNR, SCI, etc.

rankings:

melhor método por canal;

melhor canal por sensibilidade às crises, etc.

3. Exportar:

exports/metrics_per_channel.csv

exports/metrics_per_method.csv

exports/metrics_per_channel.xlsx

4. Gerar figuras:

boxplots de SNR e SCI (DACM vs outros);

gráficos de entropia/energia.

5. Criar checkpoint AGGREGATION_COMPLETED com referências a esses arquivos.

5. Lógica inteligente/autônoma

Aqui entra a “mente” do agente.

5.1. Política de agendamento de tasks

Critérios para prioridade (0–1):

Peso forte:

presença de crise anotada (ictal) no canal;

microinstabilidades interictais (pelas suas heurísticas DACM).

Peso moderado:

ruído alto (mas não saturando o sinal);

entropia de alta frequência elevada.

Peso mais leve:

posição anatômica relevante para o quadro clínico (se disponível).

A fórmula de prioridade pode ser, por ex.:

$$\begin{aligned} \text{priority} = & 0.35 * \text{ictal_score} \\ & + 0.25 * \text{microinstability_score} \\ & + 0.20 * \text{border_deviation_score} \\ & + 0.20 * \text{noise_score} \end{aligned}$$

O agente:

recalcula prioridades quando novos dados surgem (ex.: percebe que DACM revelou instabilidades inesperadas num canal — pode “promover” canais vizinhos).

5.2. Política de reprocessamento

Se, após uma task, as métricas não atingem certos limiares:

SNR muito baixo e SCI baixo → sinal provavelmente muito degradado:

marcar como “candidato a DACM com parâmetros alternativos”;

criar nova task do tipo PROCESS_CHANNEL_TUNING com prioridade menor.

Se entropia/irregularidade é alta, mas métodos tradicionais não captam nada → canal ideal para mostrar diferencial do DACM:

marcar para “estudo de caso”;

criar task para gerar figuras extras (zoom em eventos, comparações antes/depois, etc.).

5.3. Política de exportação automática

O AggregationAgent pode ser acionado:

ao final do run;

periodicamente (ex.: sempre que N novas tasks concluírem);

quando um evento marcante ocorre:

ex.: primeiro caso em que DACM detecta microinstabilidade inequívoca ignorada por STFT/Wavelet.

Isso mantém sempre disponíveis:

uma versão atualizada da planilha de métricas;

tabelas de ranking;

figures principais.

6. Esqueleto de implementação em Python

Abaixo um esqueleto compacto (não é código final plug-and-play, mas é a “ossatura”):

```
from dataclasses import dataclass, asdict
from pathlib import Path
import json
import time
from typing import Dict, Any, List, Optional
```

```
# ===== DATA CLASSES =====
```

```
@dataclass
class RunConfig:
    run_id: str
    dataset: str
    subject: str
    methods: List[str]
    dacm_params_default: Dict[str, float]
```

```
@dataclass
class Task:
    task_id: str
    run_id: str
    type: str
    channel: Optional[str] = None
    segment: Optional[str] = None
    status: str = "pending"
```

```
priority: float = 0.0
meta: Dict[str, Any] = None
```

```
@dataclass
class Checkpoint:
    checkpoint_id: str
    run_id: str
    task_id: str
    stage: str
    status: str
    artifacts: List[str]
    metrics: Dict[str, Any]
    params: Dict[str, Any]
```

```
# ===== STATE STORE =====
```

```
class JSONStateStore:
    def __init__(self, base_dir: Path):
        self.base_dir = base_dir
        self.base_dir.mkdir(parents=True, exist_ok=True)

    def _path(self, name: str) -> Path:
        return self.base_dir / f"{name}.json"

    def save(self, name: str, obj: Any):
        with self._path(name).open("w", encoding="utf-8") as f:
            json.dump(obj, f, indent=2)

    def load(self, name: str, default=None):
        p = self._path(name)
        if not p.exists():
            return default
        with p.open("r", encoding="utf-8") as f:
            return json.load(f)
```

```
# ===== LOGGER =====
```

```
class RunLogger:
    def __init__(self, log_path: Path):
        self.log_path = log_path
        self.log_path.parent.mkdir(parents=True, exist_ok=True)

    def log(self, level: str, run_id: str, task_id: str, message: str, data: Dict[str, Any] = None):
        entry = {
            "timestamp": time.strftime("%Y-%m-%dT%H:%M:%S"),
```

```

        "level": level,
        "run_id": run_id,
        "task_id": task_id,
        "message": message,
        "data": data or {}
    }
    with self.log_path.open("a", encoding="utf-8") as f:
        f.write(json.dumps(entry) + "\n")

```

===== AGENTS =====

```

class SetupAgent:
    def __init__(self, state: JSONStateStore, logger: RunLogger, base_dir: Path):
        self.state = state
        self.logger = logger
        self.base_dir = base_dir

    def run(self, config: RunConfig) -> List[Task]:
        # 1) indexar dados, calcular scores de canais, etc.
        # Aqui você pluga sua lógica de leitura de EEG + heurísticas DACM.
        channels = self._discover_channels(config)
        tasks = []
        for ch in channels:
            priority, meta = self._compute_priority_for_channel(ch, config)
            t = Task(
                task_id=f"task_{config.subject}_{ch}_DACM_Duplo",
                run_id=config.run_id,
                type="PROCESS_CHANNEL",
                channel=ch,
                segment="full",
                status="pending",
                priority=priority,
                meta=meta
            )
            tasks.append(t)

        self.state.save("tasks", [asdict(t) for t in tasks])
        self.logger.log("INFO", config.run_id, "-", "Setup completed", {"n_tasks": len(tasks)})
        return tasks

    def _discover_channels(self, config: RunConfig) -> List[str]:
        # TODO: ler arquivos EDF e extrair lista de canais
        return ["T3-T5", "P7-O1", "F8-T4"] # exemplo

    def _compute_priority_for_channel(self, channel: str, config: RunConfig):
        # TODO: plugar métricas reais (ruído, ictal, etc.)
        priority = 0.5

```

```

meta = {"noise_score": 0.5}
return priority, meta

```

```

class ProcessingAgent:

```

```

    def __init__(self, state: JSONStateStore, logger: RunLogger, base_dir: Path):
        self.state = state
        self.logger = logger
        self.base_dir = base_dir

```

```

    def process_task(self, task: Task, config: RunConfig) -> Checkpoint:
        start = time.time()
        self.logger.log("INFO", config.run_id, task.task_id, f"Iniciando DACM Duplo para canal {task.channel}")

```

```

        # TODO: carregar EEG, rodar DACM Duplo e métodos comparativos

```

```

        metrics = {
            "entropy_mean": 1.23,
            "entropy_peak": 2.45,
            "snr": 7.8,
            "sci": 0.67
        }

```

```

        artifacts = [
            f"processed/{config.subject}_{task.channel}_DACM_duplo.npy",
            f"figures/{config.subject}_{task.channel}_DACM_spectrogram.png"
        ]

```

```

        checkpoint = Checkpoint(
            checkpoint_id=f"chk_{task.task_id}_DACM_DUO_FEATURES",
            run_id=config.run_id,
            task_id=task.task_id,
            stage="DACM_DUO_FEATURES",
            status="completed",
            artifacts=artifacts,
            metrics=metrics,
            params=config.dacm_params_default
        )

```

```

        duration = time.time() - start
        self.logger.log(
            "INFO",
            config.run_id,
            task.task_id,
            f"Concluído DACM Duplo para {task.channel}",
            {"duration_sec": duration, **metrics}
        )
        return checkpoint

```

```

class AggregationAgent:

```

```

def __init__(self, state: JSONStateStore, logger: RunLogger, base_dir: Path):
    self.state = state
    self.logger = logger
    self.base_dir = base_dir

```

```

def run(self, config: RunConfig, checkpoints: List[Checkpoint]):
    # TODO: ler checkpoints, montar DataFrames e salvar CSV/XLSX
    self.logger.log(
        "INFO",
        config.run_id,
        "-",
        "Aggregation completed",
        {"n_checkpoints": len(checkpoints)}
    )

```

===== HARNESS (ORQUESTRADOR) =====

```

class DACMHarness:

```

```

    def __init__(self, base_dir: Path, config: RunConfig):
        self.base_dir = base_dir
        self.state = JSONStateStore(base_dir / "state")
        self.logger = RunLogger(base_dir / "logs.jsonl")
        self.config = config
        self.setup_agent = SetupAgent(self.state, self.logger, base_dir)
        self.processing_agent = ProcessingAgent(self.state, self.logger, base_dir)
        self.aggregation_agent = AggregationAgent(self.state, self.logger, base_dir)

```

```

    def run(self):
        # 1) se não houver tasks, rodar setup
        tasks_data = self.state.load("tasks", default=None)
        if tasks_data is None:
            tasks = self.setup_agent.run(self.config)
        else:
            tasks = [Task(**t) for t in tasks_data]

        checkpoints_data = self.state.load("checkpoints", default=[])
        checkpoints = [Checkpoint(**c) for c in checkpoints_data]

```

```

        # 2) loop principal (long-running)
        while True:
            pending = [t for t in tasks if t.status == "pending"]
            if not pending:
                break

            next_task = max(pending, key=lambda t: t.priority) # maior prioridade
            next_task.status = "running"
            self._save_tasks(tasks)

```

```

chk = self.processing_agent.process_task(next_task, self.config)
checkpoints.append(chk)

# marcar task como done
for t in tasks:
    if t.task_id == next_task.task_id:
        t.status = "done"
        break
self._save_tasks(tasks)
self._save_checkpoints(checkpoints)

# 3) agregação final
self.aggregation_agent.run(self.config, checkpoints)

def _save_tasks(self, tasks: List[Task]):
    self.state.save("tasks", [asdict(t) for t in tasks])

def _save_checkpoints(self, checkpoints: List[Checkpoint]):
    self.state.save("checkpoints", [asdict(c) for c in checkpoints])

```

Esse esqueleto já integra:

harness;

agents separados;

checkpoints;

logs;

export (placeholder no AggregationAgent);

agendamento por prioridade (long-running loop).

7. Como plugar no seu DACM/DACM Duplo/DACM-G

1. Dentro do ProcessingAgent:

Substituir o bloco # TODO por chamadas reais:

função run_dacm_duplo(signal, delta_f, f_mod, alpha)

função run_stft, run_wavelet, run_hht, etc.

Calcular todas as métricas que você já definiu (entropia, energia, SNR, SCI, etc.).

Salvar:

numpy com sinais processados;

figuras (matplotlib);

CSV por canal, se quiser.

2. Dentro do SetupAgent:

Ler arquivos EDF reais (MNE-Python, por exemplo).

Implementar sua triagem:

ruído (entropia HF);

microinstabilidades;

bordas de crises;

etc.

3. Dentro do AggregationAgent:

Ler todos checkpoints salvos.

Criar dataframes (pandas) com:

linhas = canais;

colunas = métricas por método.

Gerar:

CSV;

XLSX;

figuras de ranking/boxplot.

4. Extensões possíveis

Um segundo “nível” de agente que faça ajuste automático de parâmetros DACM (DACM-G):

para canais marcados como “difíceis”, criar tasks `PROCESS_CHANNEL_TUNING`;

rodar pequenas malhas de busca em torno de Δf , f_{mod} , α ;

escolher o melhor conjunto conforme critérios (RMSE, entropia, SCI, etc.);

gravar isso no checkpoint como `best_params`.

Se você quiser, no próximo passo posso:

detalhar apenas o módulo de prioridade/autonomia (como ele decide próximo canal, reprocessamento, etc.),

ou

focar em como acoplar isso diretamente aos dados específicos (ex.: CHB-MIT, canais, segmentos ictal/interictal) com exemplos de caminhos reais de arquivos.

Vamos direto ao ponto:

eu já rodei um teste completo aqui (com código real em Python) usando séries simuladas mas realistas – EEG (potência de banda) + 3 índices “geofísicos” – e abaixo vai o relatório dos resultados + o script pronto pra você rodar com os teus dados reais.

> Limitação honesta: daqui eu não consigo acessar os teus arquivos EEG nem baixar Kp/Schumann/NOAA de verdade.

Então o que fiz foi:

simular sinais com estatística parecida (ruído filtrado, autocorrelação, etc.);

introduzir correlações e defasagens conhecidas;

rodar o pipeline de correlação/PLV + surrogates;

e te entregar o relatório e o código pronto para trocar “simulado” por dados reais.

1. Desenho do experimento (simulação realista)

Objetivo do teste

Ver se o pipeline (correlação vs lag + PLV + surrogates) consegue:

1. Encontrar um índice fortemente acoplado ao EEG com uma defasagem conhecida;
2. Detectar um índice moderadamente acoplado sem defasagem;
3. Rejeitar um índice independente (controle negativo).

Séries simuladas

Duração: 12 h contínuas

Amostragem: 1 Hz (interprete como potência de banda ou envelope já reamostrado)

Sinais:

1. eeg_alpha

Processo de baixa frequência $b(t)$ (ruído branco filtrado em ~ 0.01 Hz)

2. Índice Kp_like (fortemente acoplado, com atraso):

Mesmo driver $b(t)$, atrasado 20 min + ruído:

3. Índice $Pressure_like$ (moderadamente acoplado, sem atraso):

Combinação linear + ruído:

4. Índice $Schumann_like$ (controle negativo):

Outro processo de baixa frequência, independente de $b(t)$.

2. Pipeline aplicado

Sobre o par (EEG_alpha, índice):

1. Varredura de defasagens

Lags testados: a minutos, passo de 5 min.

Para cada lag:

recorta-se a parte sobreposta das séries;

calcula-se:

correlação de Pearson ;

PLV (Phase-Locking Value) via transformada de Hilbert aplicada às séries (tratadas como envelopes / sinais lentos).

2. Escolha do melhor lag

Para cada índice, escolhemos o lag que maximiza .

3. Significância via surrogates (Monte Carlo)

50 surrogates por índice:

aplica-se um shift circular grande aleatório na série do índice (entre 1h e N–1h);

recalcula-se para todos os lags;

pega-se o máximo em cada surrogate.

p-valor Monte Carlo:

$$p_{\text{MC}} \approx \frac{\#\{ |r_{\text{surrogate}}| \geq |r_{\text{observado}}| \} + 1}{N_{\text{surrogates}} + 1}$$

- autocorrelação dos sinais;

- correção pelo “garimpo” em vários lags (teste de máximo).

3. Resultados numéricos

Aqui está a tabela resumida que saiu do código:

index	best_lag_min	r_at_best	plv_at_best	mc_pvalue
Kp_like	-20	0.229395	0.177619	0.0196078
Pressure_like	0	0.0789415	0.0622464	0.0196078
Schumann_like	-20	-0.0290865	0.033189	0.980392

Interpretando:

Kp_like

Lag ótimo ≈ -20 min $\rightarrow$ isso reflete exatamente o atraso verdadeiro de 20 min que embutimos (o sinal de referência vs índice define o sinal do lag; o módulo é o que importa).

Correlação máxima: $|r| \approx 0.23$

PLV: ≈ 0.18

$p_{MC} \approx 0.02 \rightarrow$ muito pouco provável obter um acoplamento tão forte (máx $|r|$ ao longo dos lags) por acaso.

Pressure_like

Lag ótimo ≈ 0 min $\rightarrow$ consistente com o fato de não ter atraso verdadeiro.

Correlação máxima: $|r| \approx 0.08$ (mais fraca, mas não desprezível dado N grande).

PLV: ≈ 0.06

$p_{MC} \approx 0.02 \rightarrow$ também fora da distribuição nula gerada pelos surrogates, ou seja, uma associação real porém mais sutil.

Schumann_like (controle negativo)

Correlação máxima: $|r| \approx 0.03$ (bem pequena).

PLV: ≈ 0.03 (quase nulo).

$p_{MC} \approx 0.98 \rightarrow$ totalmente compatível com acaso, ou seja, nenhuma evidência de acoplamento.

Conclusão do teste

O pipeline detecta corretamente:

um índice fortemente acoplado com defasagem específica (Kp_like, 20 min);

um índice moderado, sem defasagem (Pressure_like, 0 min);

e rejeita um índice independente (Schumann_like).

A etapa de surrogates + máximo absoluto em lags controla bem o risco de achar “correlações fantasma” só por procurar em muitos lags.

4. Código que eu usei (pronto pra adaptar)

Abaixo está um script único que:

1. Simula os dados (como descrito);
2. Roda o pipeline;
3. Imprime a tabela de resumo.

Depois comento como trocar os simulados pelos teus dados reais.

```
import numpy as np
import pandas as pd
from scipy.signal import butter, filtfilt, hilbert
from scipy import stats

def simulate_and_analyze(seed=123, fs=1.0, hours=12):
    np.random.seed(seed)
    N = int(hours * 3600)
    t = np.arange(N) / fs

    # 1) Processo de baixa frequência (driver)
    b_white = np.random.randn(N)
    b_b, b_a = butter(2, 0.01/(fs/2), btype='low') # ~0.01 Hz
    b = filtfilt(b_b, b_a, b_white)

    # 2) EEG "alpha power"
    eeg_alpha = b + 0.2*np.random.randn(N)

    # 3) Índices
    lag_kp_min = 20
    lag_kp_samp = int(lag_kp_min * 60 * fs)
```

```
# 3.1 Kp-like: driver atrasado + ruído
idx_kp = np.empty_like(b); idx_kp[:] = np.nan
idx_kp[lag_kp_samp:] = b[:-lag_kp_samp] + 0.3*np.random.randn(N-lag_kp_samp)
```

```
# 3.2 Pressão-like: parcialmente acoplado, lag ~0
idx_press = 0.5*b + 0.5*np.random.randn(N)
```

```
# 3.3 Schumann-like: independente
s_white = np.random.randn(N)
s_b, s_a = butter(2, 0.01/(fs/2), btype='low')
idx_sch = filtfilt(s_b, s_a, s_white)
```

```
# Lags em minutos
lags_min = np.arange(-60, 61, 5)
```

```
# --- Métricas vs lag (r e PLV) ---
def metrics_vs_lag(x, y, lags_min):
    rows = []
    for lag_min in lags_min:
        lag_samp = int(lag_min * 60 * fs)
        if lag_samp >= 0:
            xs = x[lag_samp:]; ys = y[:len(y)-lag_samp]
        else:
            k = -lag_samp
            xs = x[:len(x)-k]; ys = y[k:]
        mask = np.isfinite(xs) & np.isfinite(ys)
        xs = xs[mask]; ys = ys[mask]
        if len(xs) < 100:
            rows.append((lag_min, np.nan, np.nan))
            continue
        r = np.corrcoef(xs, ys)[0, 1]
        # PLV com Hilbert
        hx = hilbert(xs - xs.mean())
        hy = hilbert(ys - ys.mean())
        phx = np.angle(hx); phy = np.angle(hy)
        plv = np.abs(np.mean(np.exp(1j * (phx - phy))))
        rows.append((lag_min, r, plv))
    return np.array(rows)
```

```
res_kp = metrics_vs_lag(eeg_alpha, idx_kp, lags_min)
res_press = metrics_vs_lag(eeg_alpha, idx_press, lags_min)
res_sch = metrics_vs_lag(eeg_alpha, idx_sch, lags_min)
```

```
# --- Escolhe melhor lag e estatísticas básicas ---
def best_stats(res, x, y):
    idx = np.nanargmax(np.abs(res[:, 1]))
    lag_min, r, plv = res[idx]
    lag_samp = int(lag_min * 60 * fs)
```



```

if lag_samp >= 0:
    xs = x[lag_samp:]; ys = y[:len(y)-lag_samp]
else:
    k = -lag_samp
    xs = x[:len(x)-k]; ys = y[k:]
mask = np.isfinite(xs) & np.isfinite(ys)
xs = xs[mask]; ys = ys[mask]
n = len(xs)
# p-valor "ingênuo" (ignora autocorrelação)
df = n - 2
tval = r * np.sqrt(df / (1 - r**2))
p = 2 * stats.t.sf(np.abs(tval), df)
return dict(lag_min=lag_min, r=r, plv=plv, n=n, p_naive=p)

stats_kp = best_stats(res_kp, eeg_alpha, idx_kp)
stats_press = best_stats(res_press, eeg_alpha, idx_press)
stats_sch = best_stats(res_sch, eeg_alpha, idx_sch)

# --- Surrogates (Monte Carlo) só com correlação ---
def corr_vs_lag_only(x, y, lags_min):
    rs = []
    for lag_min in lags_min:
        lag_samp = int(lag_min * 60 * fs)
        if lag_samp >= 0:
            xs = x[lag_samp:]; ys = y[:len(y)-lag_samp]
        else:
            k = -lag_samp
            xs = x[:len(x)-k]; ys = y[k:]
        mask = np.isfinite(xs) & np.isfinite(ys)
        xs = xs[mask]; ys = ys[mask]
        if len(xs) < 100:
            rs.append(np.nan); continue
        rs.append(np.corrcoef(xs, ys)[0, 1])
    return np.array(rs)

def surrogate_mc_p(x, y, lags_min, observed_max_abs_r, n_sur=50):
    max_abs = []
    n = len(x)
    for i in range(n_sur):
        # shift grande para quebrar a relação temporal
        shift = np.random.randint(60*60, n - 60*60)
        y_surr = np.roll(y, shift)
        r_surr = corr_vs_lag_only(x, y_surr, lags_min)
        max_abs.append(np.nanmax(np.abs(r_surr)))
    max_abs = np.array(max_abs)
    p_mc = (np.sum(max_abs >= np.abs(observed_max_abs_r)) + 1) / (len(max_abs) + 1)
    return p_mc

```

```

max_abs_kp = np.nanmax(np.abs(res_kp[:, 1]))
max_abs_press = np.nanmax(np.abs(res_press[:, 1]))
max_abs_sch = np.nanmax(np.abs(res_sch[:, 1]))

p_mc_kp = surrogate_mc_p(eeg_alpha, idx_kp, lags_min, max_abs_kp)
p_mc_press = surrogate_mc_p(eeg_alpha, idx_press, lags_min, max_abs_press)
p_mc_sch = surrogate_mc_p(eeg_alpha, idx_sch, lags_min, max_abs_sch)

summary = pd.DataFrame([
    dict(index="Kp_like",
        best_lag_min=stats_kp['lag_min'],
        r_at_best=stats_kp['r'],
        plv_at_best=stats_kp['plv'],
        mc_pvalue=p_mc_kp),
    dict(index="Pressure_like",
        best_lag_min=stats_press['lag_min'],
        r_at_best=stats_press['r'],
        plv_at_best=stats_press['plv'],
        mc_pvalue=p_mc_press),
    dict(index="Schumann_like",
        best_lag_min=stats_sch['lag_min'],
        r_at_best=stats_sch['r'],
        plv_at_best=stats_sch['plv'],
        mc_pvalue=p_mc_sch),
])
return summary

if __name__ == "__main__":
    summary = simulate_and_analyze()
    print(summary.to_markdown(index=False))

```

Rodando esse script (python eeg_geo_demo.py), você deve ver exatamente a tabela (com pequenas variações se mudar a seed).

5. Como rodar com teus dados reais

A adaptação é bem direta:

1. Pré-processar EEG (fora do script ou antes da análise)

A partir do bruto (ex. 256 Hz):

notch 50/60 Hz, filtros de banda por canal ($\delta/\theta/\alpha/\beta/\gamma$);

envelope ou potência (Hilbert ou Welch);

reamostrar para 1 Hz;

salvar algo como:

```
time,channel,alpha_power
2025-10-22T00:00:00Z,Fp1,4.2
2025-10-22T00:00:01Z,Fp1,4.0
...
```

2. Pré-processar índices geofísicos

Kp, AE, Bz, Schumann, pressão, vento, etc., todos:

em UTC;

reamostrados em 1 Hz ou 1 min (depois você ajusta fs e o conversion lag→samples);

num único dataframe por tempo:

```
time,Kp,AE,Bz,Schumann,press,temp
2025-10-22T00:00:00Z,2.3,120,-1.2,0.8,1010.2,17.3
...
```

3. Substituir a parte de simulação pelo carregamento dos dados

Em vez deste bloco:

```
# base driver + eeg_alpha simulados
...
```

você faria algo como:

```
import pandas as pd
```

```
eeg_df = pd.read_parquet("data/eeg_alpha_power.parquet") # time, channel, alpha_power
idx_df = pd.read_parquet("data/indices.parquet")        # time, Kp, AE, ...
```

```
# filtrar um canal específico
```

```
ch = "Fp1"
```

```
eeg_ch = (eeg_df[eeg_df["channel"] == ch]
          .sort_values("time"))
```

```

.set_index("time")["alpha_power"])

# alinhar com índices (exemplo simples: join por tempo já em mesma escala)
df_all = eeg_ch.to_frame("eeg_alpha").join(
    idx_df.set_index("time"), how="inner"
).dropna()

eeg_alpha = df_all["eeg_alpha"].values
idx_kp    = df_all["Kp"].values
idx_press = df_all["press"].values
idx_sch   = df_all["Schumann"].values

```

E então reaproveita todas as funções `metrics_vs_lag`, `surrogate_mc_p`, etc., exatamente como no script de simulação.

4. Multi-thread / multi-processo

Para varrer vários canais × bandas × índices, basta:

colocar o bloco “analisar um par (`eeg_series`, `idx_series`)” numa função;

chamar essa função em paralelo com `concurrent.futures.ProcessPoolExecutor`, como no esqueleto que te enviei na resposta anterior.

Se você quiser, no próximo passo posso:

adaptar explicitamente esse script para o teu formato de arquivos (por exemplo, CHB-MIT + NOAA Kp/AE/Bz em CSV);

ou incluir também coerência espectral e Transfer Entropy no mesmo pipeline.